Tecnicatura Universitaria en Programación - A Distancia

UTN - Trabajo Práctico Integrador: Programación 1

Sistema de Control de Inventario

Informe de Diseño y Fundamentación

1. Conceptos Aplicados

Listas

Una **lista** es una estructura de datos mutable que permite almacenar colecciones ordenadas de elementos. En este proyecto, se utilizó una lista principal llamada `inventario`, la cual se inicializa vacía al comienzo de `main.py`. Esta lista actúa como la base de datos temporal del sistema, almacenando todos los productos cargados. Para cumplir con las buenas prácticas y evitar el uso de variables globales, la lista se pasa por parámetro a cada una de las funciones del módulo `operaciones.py` cada vez que se requiere consultar o modificar un dato.

Diccionarios

Un diccionario es una estructura que almacena datos en pares de clave-valor, ideal para representar lógicamente objetos del mundo real. En el sistema, cada herramienta se modeló como un diccionario independiente con exactamente dos claves: 'herramienta' (almacena el nombre validado como cadena de texto) y `'cantidad'` (almacena el stock numérico entero). Estos diccionarios se instancian y se agregan directamente a la lista principal mediante inventario.append() en las funciones de alta, sin utilizar diccionarios globales sueltos.

Funciones

Las **funciones** son bloques de código reutilizables que encapsulan una tarea específica, fundamentales para la modularidad. En lugar de escribir un bloque principal monolítico, el sistema delegó toda la lógica de negocio al archivo `operaciones.py`. Cada opción del menú interactivo (como `consultar_stock` o `actualizar_stock`) se aisló en su propia función definida con `def`. Esto permitió que `main.py` actúe únicamente como orquestador, facilitando el mantenimiento y la delegación de excepciones.

Condicionales

Las **estructuras condicionales** (`if`/`elif`/`else`) se aplicaron para el control de flujo y la validación estricta de las reglas de negocio. En `main.py`, se usaron para direccionar la opción numérica elegida hacia la función correspondiente. En `operaciones.py`, se utilizaron múltiples sentencias `if` para proteger el sistema: por ejemplo, validar que los nombres no estén vacíos (if not herramienta:), comprobar de forma anticipada si el inventario carece de elementos para abortar procesos tempranamente (salida temprana), y evitar que un ingreso de stock sea negativo.

Bucles (Ciclos)

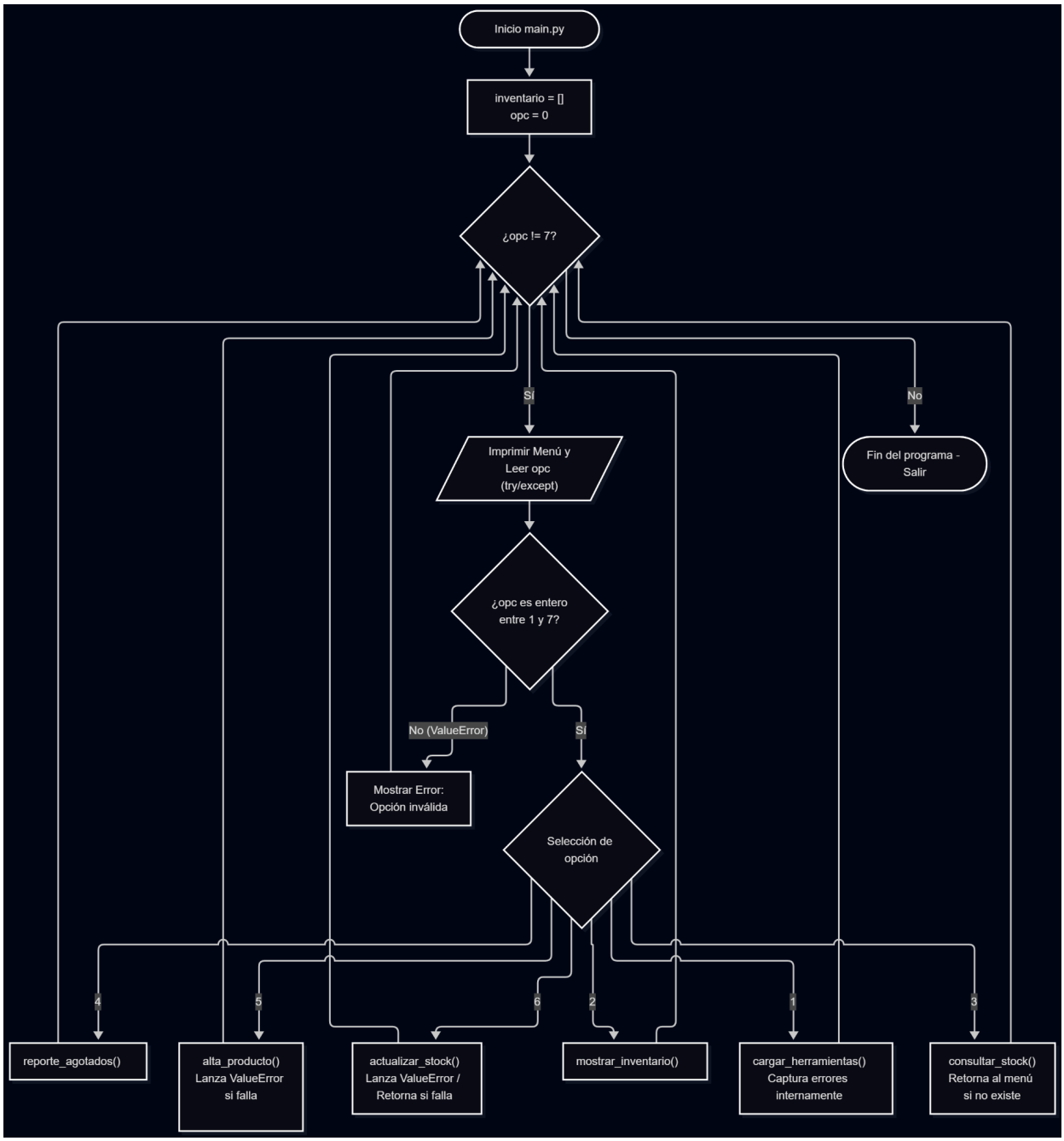
Los **bucles** son esenciales para la repetición de tareas y la persistencia de la interfaz. En main.py, se implementó un while opc != 7: para mantener el menú activo de forma persistente tras cada operación. En `operaciones.py`, el ciclo `for producto in inventario:` constituyó la base de todas las búsquedas secuenciales, permitiendo detectar productos duplicados durante el alta o localizar herramientas para actualizar su stock. Asimismo, en la Opción 1, se empleó while True para retener al usuario en un campo específico hasta que ingresara un dato válido, sin perder el progreso de la carga.

Manejo de Errores (try / except / raise)

Se adoptó una arquitectura robusta orientada a delegar responsabilidades ("Fail Fast"). La palabra reservada `raise ValueError(...)` se utilizó para hacer explícitas las violaciones a las reglas de negocio (ej. nombres vacíos, duplicados, stock negativo). Se implementó `try`/`except` globalmente en `main.py` para capturar errores de conversión (cuando se ingresan caracteres alfabéticos en un campo numérico como `int()`) y para atajar de forma centralizada las excepciones delegadas por las opciones 3, 5 y 6. En

contraste, en la opción 1, el try/except se usó de forma local dentro del ciclo para permitir reintentos parciales.

2. Diagrama de Flujo



3. Lenguaje, Entorno y Recursos

Lenguaje: Se utilizó Python 3.x. Fue elegido por su legibilidad, su eficiente manejo nativo de estructuras de datos como diccionarios y listas, y por ser el estándar requerido por la cátedra para aplicar lógicas de programación estructurada.
Entorno de Desarrollo: Se empleó Visual Studio Code (VS Code). Este IDE fue seleccionado por su terminal integrada, que facilita la ejecución del menú interactivo, y por su robusto soporte de linting y control de versiones nativo.
Control de Versiones: Se utilizó Git junto con GitHub como repositorio remoto. Esto permitió mantener un historial ordenado del desarrollo (commits semánticos) y versionar el código a medida que se implementaban y refactorizaban las funciones de la rúbrica.
Ejecución del Sistema: El proyecto ha sido modularizado. El archivo que debe ejecutarse para iniciar el sistema es `main.py`, el cual importa las funciones correspondientes desde `operaciones.py`.